

EduPredict

Student Performance Analytics & Prediction

System

Project

Report

Course: Fundamentals of AI and ML
(Flipped Course Evaluation)

Platform: VITyarthi

Name: Joudi Chandrahas Reddy (25MIM10043)

2. Introduction

For this project I built EduPredict, a command-line tool that tries to guess how a student is going to do (pass or fail, and roughly what final score they'll land on) using four things that are easy to collect early in a term: how many hours a day they study, their attendance percentage, their prior exam score, and how many assignments they've turned in. I picked this topic because it's a fairly natural fit for the "Fundamentals of AI and ML" syllabus. It needed a real classification problem, a real regression problem, an honest evaluation step, and a bit of software engineering around all of that so it isn't just a notebook full of print statements.

I generate my own training data instead of pulling a dataset from Kaggle (more on why in Section 8), train two scikit-learn RandomForest models on it, evaluate them properly, and wrap the whole thing in a small CLI application with a SQLite database, logging, and unit tests behind it. The assignment wanted something that actually runs from a terminal and not just a

script that only works on my machine.

3. Problem Statement

The idea started from something fairly obvious once you think about it. By the time a teacher realises a student is struggling, it's usually because that student already failed a test or missed a deadline, and at that point there isn't much time left to actually help them. But the warning signs are usually visible way earlier than that: how much someone is studying, whether they're showing up to class, how they did on earlier tests, whether they're keeping up with assignments. All of that is available well before the final result comes in.

EduPredict is my attempt at turning those signals into an early prediction, so that if a student is trending toward a fail, a mentor or instructor (or the student themselves) has some warning while there's still time to do something about it, instead of finding out at the end of the semester.

4. Functional Requirements

I split the system into three functional modules. Each one does one job and doesn't reach into the others' internals:

- **Data management:** basic CRUD (add / view / update / delete) for student records, stored in a local SQLite database. Bad input (empty name, attendance over 100, etc.) gets rejected with a clear error instead of silently corrupting the table.
- **Prediction engine:** this is the actual ML part. It builds a training dataset, trains a RandomForest classifier for pass/fail and a RandomForest regressor for the numeric score, saves both models to disk, and exposes a predict() call the CLI uses.
- **Analytics & reporting:** once there's data in the database, this module works out class averages and pass rate, and saves a few chart images (score spread, which features mattered most, study hours vs. score) plus a short text summary.

Roughly, the flow goes: get student data in (typed in one at a time, or seeded in bulk) →

train the models on that kind of data → ask for a prediction on a new student → look at the analytics/report to see the bigger picture. Section 7.2 has a diagram of this.

5. Non-Functional Requirements

The brief asked for at least four; I ended up covering these:

Requirement	How I handled it
Performance	Training on 800 synthetic samples completes in under 2 seconds; predictions on persisted models are near-instant.
Reliability	All database writes run inside transactions with automatic rollback on failure.
Usability	A single, discoverable CLI with --help on every command and readable tabular output.
Maintainability	Each module has one responsibility; modules communicate through small, well-defined function signatures rather than shared global state.
Scalability	SQLite + RandomForest comfortably handle tens of thousands of records for this workload; the storage and model layers are isolated so either could be swapped out.

Security	All external input is validated before it reaches the database or model layer, preventing malformed or out-of-range values from being persisted.
----------	--

Logging / Monitoring	Every CRUD operation, training run, and prediction is logged with a timestamp to reports/edupredict.log.
Error handling strategy	A dedicated exception hierarchy (ValidationError, RecordNotFoundError, ModelNotTrainedError) lets the CLI distinguish user mistakes from system faults and report clear messages instead of raw tracebacks.

6. System Architecture

The architecture is pretty simple on purpose. A thin CLI layer at the top just parses whatever command you typed and hands it off to whichever module handles it: database, ML, or analytics. Those three don't call each other directly; they all lean on a shared utilities module for logging and error types, but otherwise they're independent. Each one owns its own bit of storage too (the SQLite file, the models/ folder, the reports/ folder) so nothing steps on anything else.

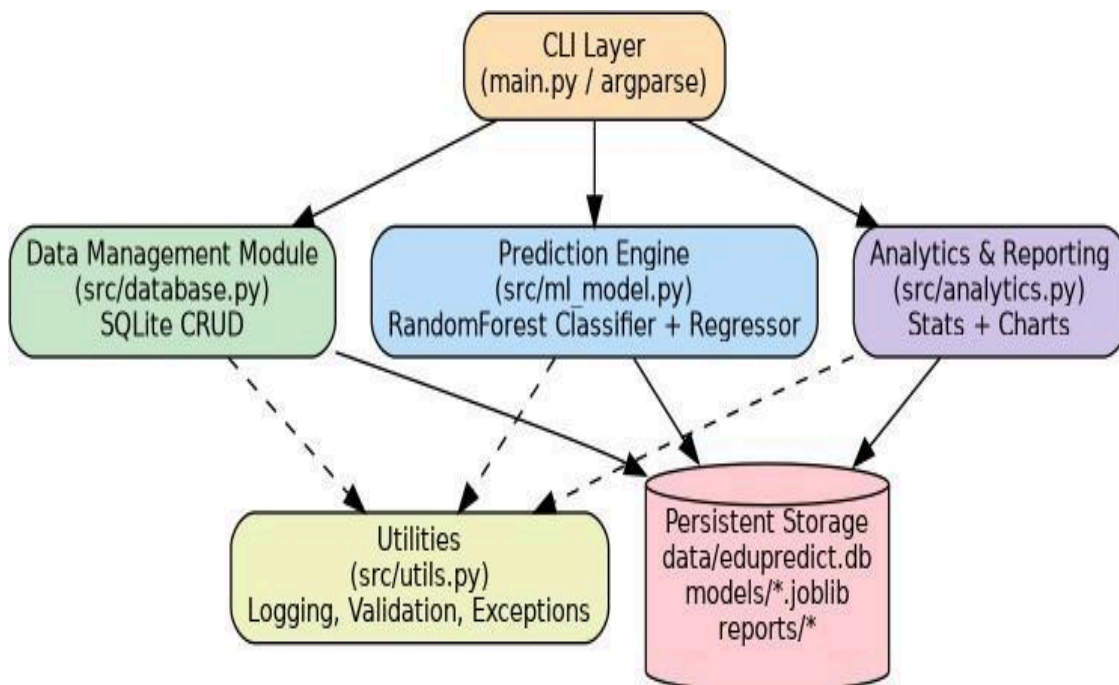


Figure 6.1 -- System Architecture Diagram

7. Design Diagrams

7.1 Use Case Diagram

Whoever is using this (student, instructor, or whoever's evaluating it) interacts with it through eight commands, shown here as use cases.

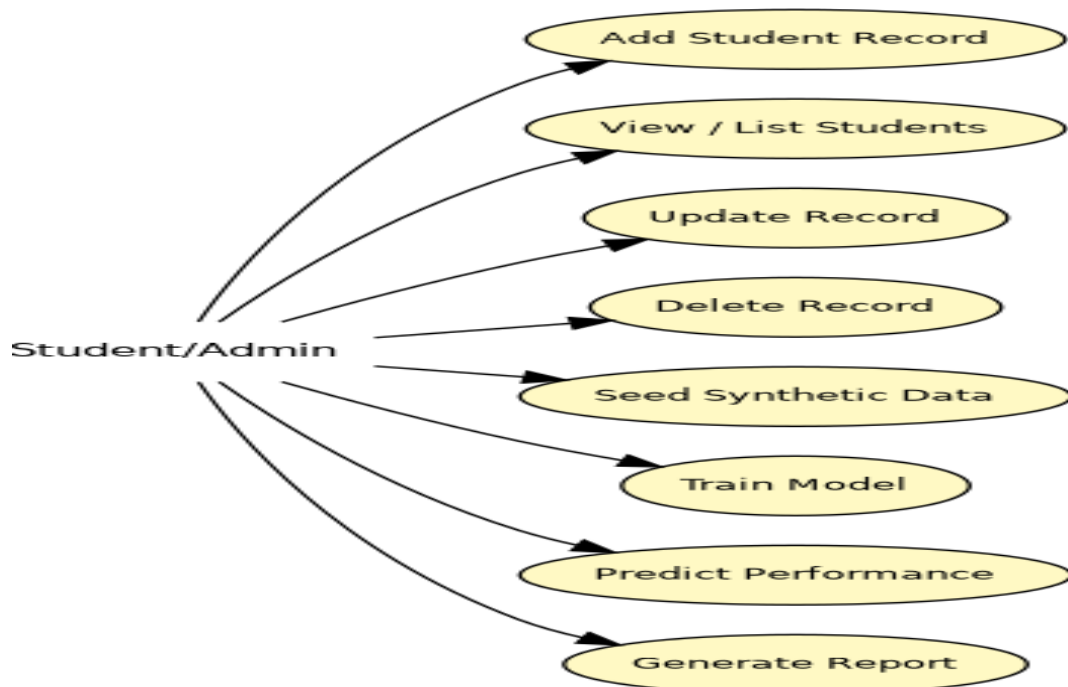


Figure 7.1 -- Use Case Diagram

7.2 Workflow Diagram

This is the order things actually happen in, from putting data in to getting a report out the other end.

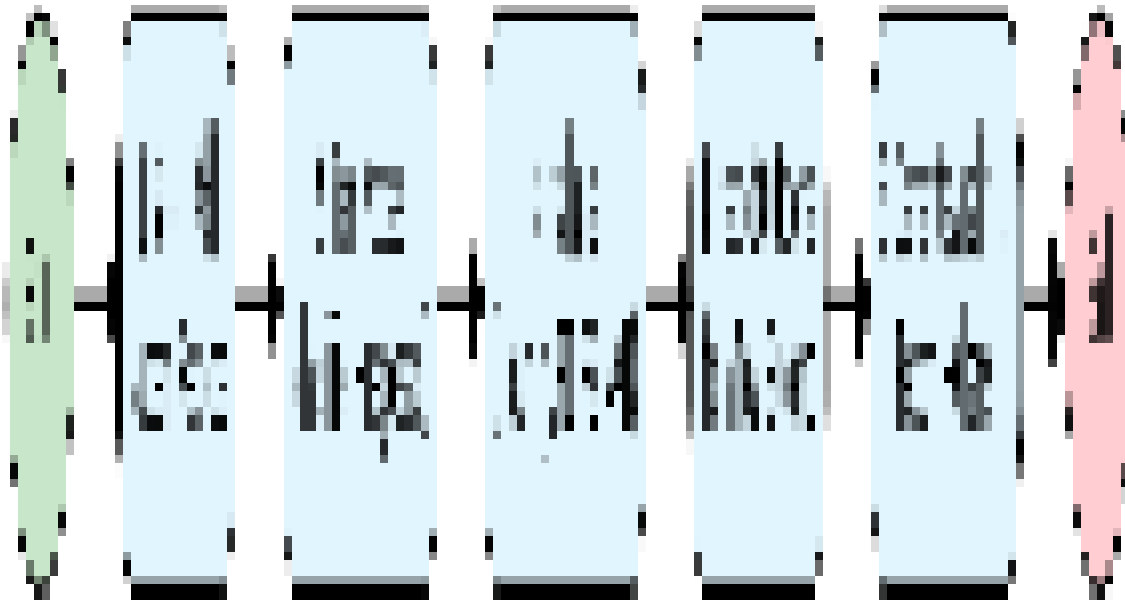


Figure 7.2 -- Process Workflow Diagram

7.3 Sequence Diagram (Prediction Flow)

I picked the predict command for this one since it's the operation people will run most, and it touches every layer of the app.

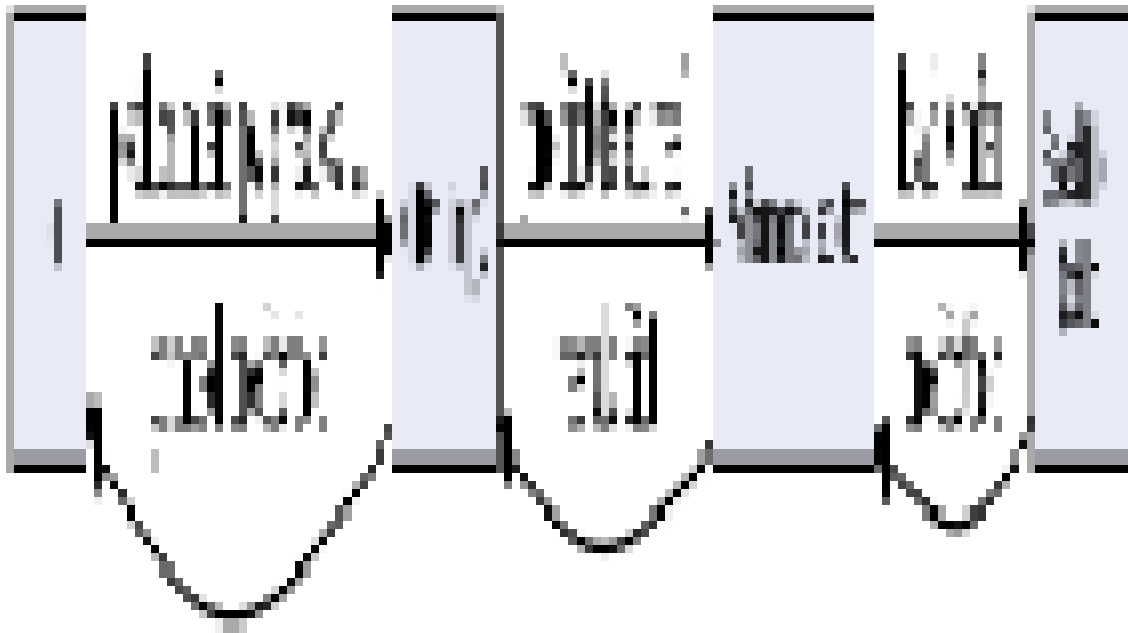


Figure 7.3 -- Sequence Diagram: Predict Command

7.4 Class / Component Diagram

The main classes/modules and how they depend on each other.

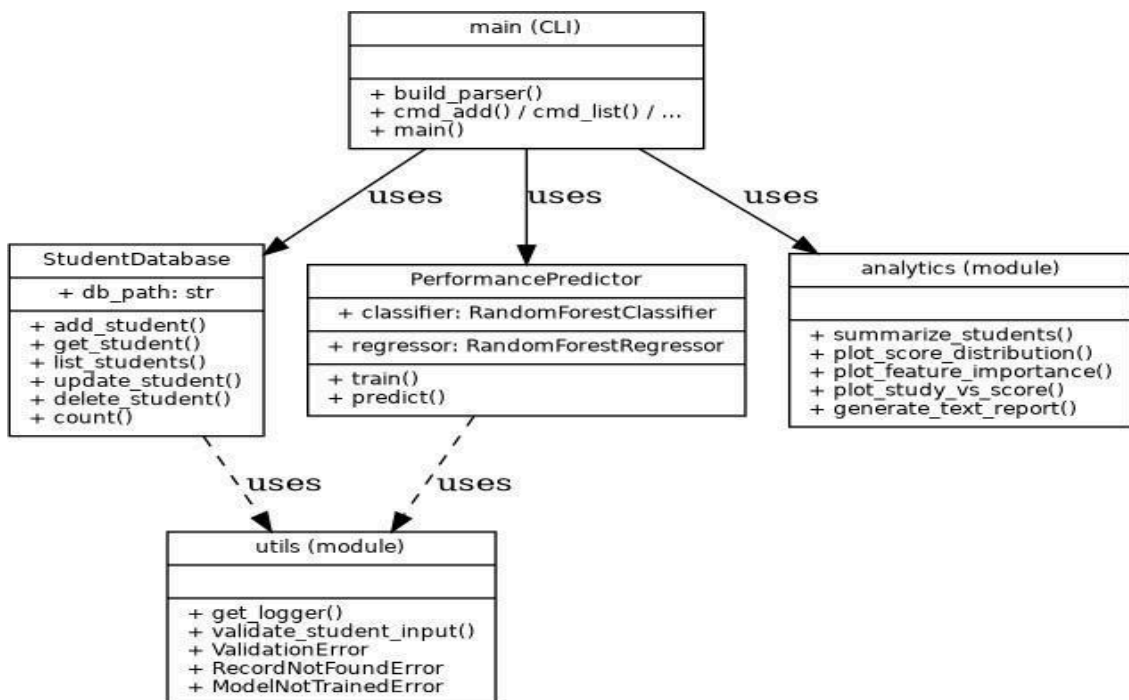


Figure 7.4 -- Class / Component Diagram

7.5 ER Diagram (Database Schema)

I kept this to a single students table. There wasn't really a need to normalise anything further for a project this size. final_score and passed start out empty and get filled in later, either manually or by the model.

students	id : INTEGER (PK) name : TEXT study_hours_per_day : REAL attendance_percentage : REAL prior_exam_score : REAL assignments_submitted : INTEGER final_score : REAL (nullable) passed : INTEGER (nullable) created_at : TEXT
----------	---

Figure 7.5 -- Entity-Relationship Diagram

8. Design Decisions & Rationale

A few decisions worth explaining, since they weren't the only options on the table:

- **Why SQLite:** I didn't want anyone grading this to have to spin up a database server just to run a student project. SQLite is a single file, ships with Python, and that's basically it.
- **Why a synthetic dataset instead of a real one:** this was the trickiest call. I looked at a couple of public student-performance datasets on Kaggle, but most of them are either quite small, or have licensing that made me uneasy about reusing them wholesale for a course submission. So instead I wrote my own generator: study hours, attendance, prior score and assignments get combined with fixed weights (attendance and prior score matter more than the others, roughly matching what education research usually says) plus some random noise, and a 40-mark cutoff decides pass/fail. It's not real data, but the relationship it encodes is realistic and, more importantly, I know exactly what the model

should be learning, so I can check whether it actually learned it (Section 10.1).

- **Why RandomForest and not something simpler like logistic regression:** I tried a plain logistic regression first as a baseline while testing, and RandomForest did noticeably better on the same data, plus it gives you feature importances basically for free, which was useful for the sanity check mentioned above.
- **Why two separate models instead of one:** pass/fail and 'what score exactly' are different kinds of questions. One's a yes/no, the other's a number, so I trained them separately rather than trying to force one model to do both.
- **Why joblib to save the models:** retraining on every single prediction would be wasteful and slow. Train once, save to disk, load when needed.

- **Why argparse subcommands (add/list/train/predict/...) instead of one big script with flags:** it makes --help actually useful for each command, and it was closer to how I'd want to use the tool myself.

9. Implementation Details

Here's roughly what's in each file:

- **src/database.py:** the StudentDatabase class. Connections are opened through a context manager so every write either commits fully or rolls back, nothing gets left half-done.
- **src/ml_model.py:**
generate_synthetic_dataset() builds the training data (the weighted formula I mentioned above); PerformancePredictor wraps training and prediction for both models and saves them under models/ once trained.

- **src/analytics.py**: just takes plain lists/dicts in and returns stats or a saved chart path out. I kept it from importing the database or ML modules directly on purpose, which made it a lot easier to test on its own with made-up sample data.
- **src/utils.py**: logging setup (writes to both the console and reports/edupredict.log) plus three custom exceptions I use to tell apart 'user typed something wrong' from 'something in the system actually broke'.
- **main.py**: the CLI itself, eight subcommands (add, list, update, delete, seed, train, predict, report), each one just wiring the right module call to the right arguments and catching errors so they print nicely instead of dumping a traceback.

That's 5 files under src/, 1 entry point, and 3 test files. 9 Python files altogether, which covers the 5-10 modules/files the brief asked for.

10. Screenshots / Results

These charts came from running `python main.py report` after seeding 200 synthetic students and training on 800 samples (`python main.py train --n 800`).

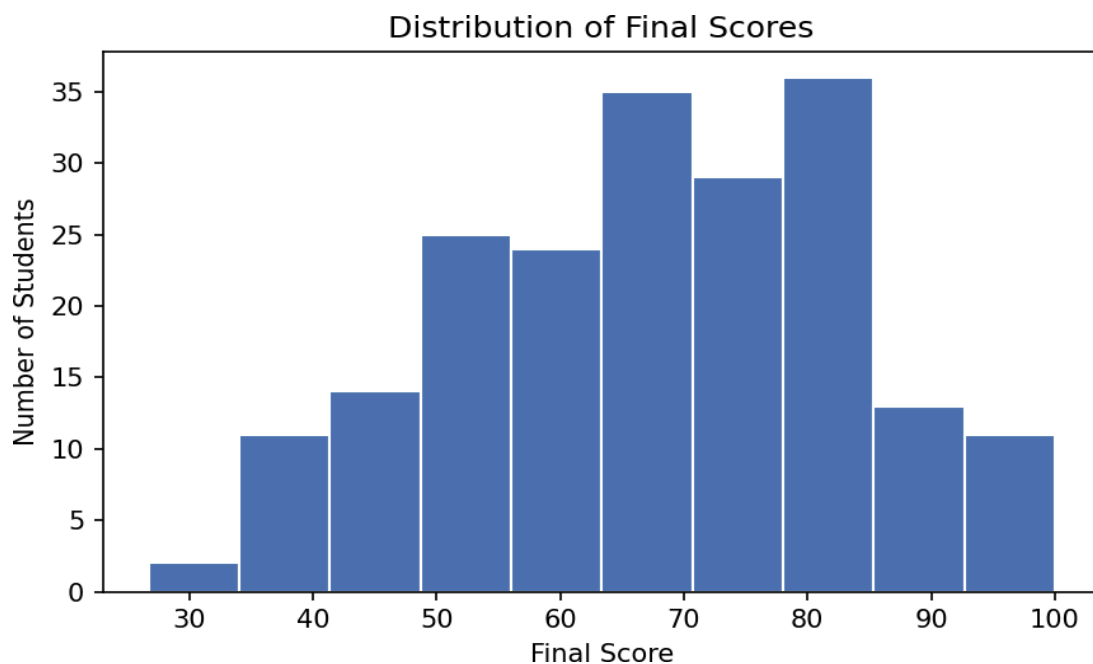


Figure 10.1 -- Distribution of Final Scores

[Diagram not found: feature_importance.png]

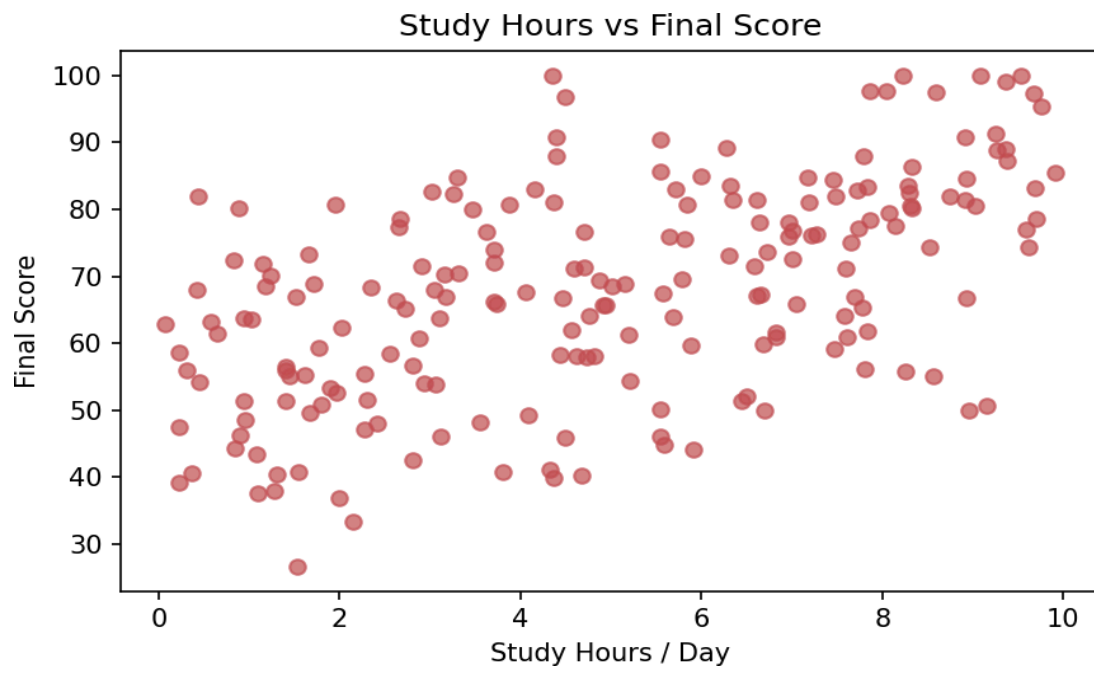


Figure 10.3 -- Study Hours vs Final Score

10.1 Model Evaluation Results (800-sample training run)

Metric	Value
Classification accuracy	97.5%
Classification precision	0.9809
Classification recall	0.9935
Classification F1-score	0.9872
Regression MAE	5.63 points
Regression R ²	0.7856

One thing I checked specifically: does the model's feature importance actually match the weights I used to generate the data? It does. attendance_percentage came out highest (0.349), then prior_exam_score (0.242), then study_hours_per_day (0.238), then assignments_submitted (0.170), which lines up with how I weighted them when building the dataset. That was a useful sanity check that the model is picking up the real pattern and not just noise.

11. Testing Approach

I used Python's built-in unittest for this rather than pulling in pytest, mostly because it needs zero extra setup for anyone running the project. 15 tests across the three modules:

- **tests/test_database.py** (7 tests): the normal add/list/update/delete paths, plus what happens when you pass bad input (empty name, attendance above 100) or ask for a student id that doesn't exist.
- **tests/test_ml_model.py** (4 tests): checks the generated dataset actually has the right shape and sane value ranges, that predicting before training raises the right error, and a full train-then-predict run where a strong student profile should come back predicted-to-pass.
- **tests/test_analytics.py** (4 tests): summary stats on empty vs. populated lists, and that the chart functions actually write a file.

Run it with `python -m unittest discover -s tests -v`. All 15 pass. The database tests each get their own temporary SQLite file so they don't touch the real data/edupredict.db I was using while testing the CLI manually.

12. Challenges Faced

A few things that took longer than I expected, or that I got wrong on the first pass:

- Getting the synthetic dataset right took a couple of tries. My first version was too clean, basically no noise, so the classifier hit close to 100% accuracy immediately, which isn't realistic and doesn't really prove anything. Adding Gaussian noise on top of the weighted formula made the problem genuinely harder and the accuracy numbers meaningful.
- I hadn't thought about matplotlib needing a GUI backend until it crashed the first time I ran the report command from a plain terminal with no display. Switching to the 'Agg' backend explicitly fixed it.
- I went back and forth on whether the analytics module should just import the database module directly to fetch students itself. Keeping it to plain function arguments instead meant a bit more wiring in main.py, but made analytics.py much easier to test without a real database.

- Deciding where to put validation (in the CLI, or in the database layer) took a moment too. I ended up putting it in `database.py` so it can't be bypassed even if something else calls `add_student()` directly later.

13. Learnings & Key Takeaways

- Going through the whole supervised learning pipeline myself, not just `fit()` and `predict()` but actually thinking about the train/test split, which metrics to report, and what those metrics were telling me, made the difference between accuracy/precision/recall/F1 (and MAE/R² for the regression side) click in a way that just reading about them didn't.
- Feature importance turned out to be a genuinely useful debugging tool, not just a nice chart to include. It's basically the fastest way to check 'is this model actually learning the right thing'.
- Splitting the project into modules that only talk to each other through plain function calls (no shared global state) made testing each piece in isolation much less painful than I expected.

- Small thing, but putting real validation and real error messages in from the start saved me a lot of confused debugging later, compared to letting bad input just crash things with a traceback.

14. Future Enhancements

If I kept working on this past the deadline, here's what I'd look at next:

- Swap the synthetic dataset for a real, anonymised one if I can get access to actual academic records, and see how the model holds up.
- Try hyperparameter tuning (grid search or similar) on the regressor specifically. An R^2 of 0.79 is decent but there's probably room to push it higher.
- Wrap the same prediction engine in a small REST API so it could sit behind a web front end later instead of only a CLI.

- Track a student over multiple points in a term instead of a single snapshot, so the model could pick up on trends (improving vs. declining) rather than just a static profile.
- Add some notion of user roles (instructor vs. student) if this ever grew beyond a single-user command-line tool.

15. References

- Pedregosa, F. et al. (2011). *Scikit-learn: Machine Learning in Python*. Journal of Machine Learning Research, 12, 2825-2830.
- Python Software Foundation. *sqlite3 -- DB-API 2.0 interface for SQLite databases*. Python 3 Standard Library Documentation.
- Hunter, J. D. (2007). *Matplotlib: A 2D Graphics Environment*. Computing in Science & Engineering, 9(3), 90-95.
- McKinney, W. (2010). *Data Structures for Statistical Computing in Python*. Proceedings of the 9th Python in Science Conference.

- Course materials: Fundamentals of AI and ML (VITyarthi flipped-course evaluation).